

MODULE 3: PROGRAMMING FOR BIOLOGICAL DATA

Introduction to Bioconductor

Session 9: Sequence Handling with Biostrings

Instructor: Dr. Haogao Gu | **Date:** February 2, 2026

Welcome to Phase III: Bioinformatics

From Statistics to Sequences

We have covered R basics and statistical testing. Now we move to handling biological data directly.

Sessions 9-12 Focus:
Sequence analysis, phylogenetics, and visualization.

Session	Topic
9	Bioconductor & Biostrings
10	Sanger Sequencing Analysis
11	Phylogenetics (MSA, Trees)
12	Visualization & Reporting

Learning Objectives



Ecosystem

Understand the
Bioconductor
repository vs CRAN.



Install

Install packages
correctly using
BiocManager.



Biostrings

Manipulate DNA,
RNA, and Amino
Acid sequences.



FASTA

Read, write, and
process FASTA files.

Part 1: Bioconductor vs. CRAN

The Bioinformatics Repository

What is Bioconductor?

The Premier Repository

Founded in 2001 by Robert Gentleman (co-creator of R).

It is distinct from CRAN (Comprehensive R Archive Network).

"Bioconductor provides tools for the analysis and comprehension of high-throughput genomic data."



[About](#) [Learn](#) [Packages](#) [Developers](#)

[Get Started >](#)

Open source software for Bioinformatics

The Bioconductor project aims to develop and share open source software for precise and repeatable analysis of biological data.
We foster an inclusive and collaborative community of developers and data scientists.

[Get started >](#)

CRAN vs Bioconductor

Feature	CRAN	Bioconductor
Focus	General Purpose Statistics	Genomics & Bioinformatics
Packages	~20,000+	~2,200 specialized
Release Cycle	Rolling (Daily)	Biannual (Spring/Fall)
Installation	<code>install.packages()</code>	<code>BiocManager::install()</code>
Objects	<code>data.frame</code> , <code>matrix</code>	<code>DNAStrng</code> , <code>GRanges</code> , <code>ExpressionSet</code>

Installing with BiocManager

Step 1: Install Manager

First, you need the tool that manages Bioconductor packages. This comes from CRAN.

```
install.packages("BiocManager")
```

Step 2: Install Packages

Use the manager to install specific tools.

```
BiocManager::install("Biostrings")  
BiocManager::install("ggtree")
```

Installation in Google Colab

⚠️ Performance Warning

Bioconductor installation compiles code, which can be slow in Colab. Always verify if the package is already installed to save time.

```
# Optimized Colab Installation
if (!requireNamespace("Biostrings", quietly = TRUE)) {
  BiocManager::install("Biostrings",
                        update = FALSE,
                        ask = FALSE)
}
```

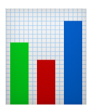
update = FALSE prevents R from trying to update all your other packages, which can take 20+ minutes.

Key Bioconductor Packages for MLTs



Biostrings

Core package for
DNA, RNA, and
Amino Acid
sequence
manipulation.



sangeranalyseR

Quality control and
processing of Sanger
sequencing
chromatograms (.ab1).



ggtree

Visualization of
phylogenetic
trees and
associated data.



GenomicRanges

Handling genomic
coordinates
(chromosomes,
start/end positions).

Biostrings Overview

The Foundation

Biostrings is the "Excel" of sequence data in R.

- Efficient storage (doesn't waste memory).
- Specialized algorithms (Pattern matching).
- File I/O (FASTA, FASTQ).

Bioconductor/ **Biostrings**

Efficient manipulation of biological strings



 13
Contributors

 19
Issues

 66
Stars

 17
Forks



Part 2: Working with Biostrings

Classes, Functions, and Operations

Why Not Regular R Strings?

Standard R (Character)

```
seq ← "ATGCATGC"
```

- No validation (can hold "Hello World").
- Memory intensive (stores each letter as a full character).
- No biological context.

Biostrings (DNASTring)

```
seq ← DNASTring("ATGCATGC")
```

- **Validation:** Only allows A, T, G, C, N, etc.
- **Memory:** Bit-encoded (2 bits per base).
- **Context:** Knows how to complement and translate.

The DNASTring Class

Used for holding a **single** sequence.

```
library(Biostrings)

# Create a DNASTring
my_seq ← DNASTring("ATGCGATCGATCG")

print(my_seq)
# 13-letter DNASTring object
# seq: ATGCGATCGATCG

length(my_seq) # Output: 13
```

The DNASTringSet Class

Used for holding **multiple** sequences (e.g., from a FASTA file).

```
seqs <- DNASTringSet(c(
  "Sample1" = "ATGATGATG",
  "Sample2" = "ATGCTGCTG",
  "Sample3" = "ATGTTTAAA"
))

print(seqs)
# DNASTringSet object of length 3:
#      width seq      names
# [1]     9 ATGATGATG Sample1
# [2]     9 ATGCTGCTG Sample2
# [3]     9 ATGTTTAAA Sample3
```

Complement Function

Biological Rule

Watson-Crick base pairing:

- **A** pairs with **T**
- **G** pairs with **C**

R Code

```
seq ← DNASTring("ATGCCC")  
  
complement(seq)  
# Output: TACGGG
```

Note: This is the complement strand reading 3' to 5', or simply the paired bases.

Reverse Complement

The sequence of the opposite strand, read 5' to 3'.

5' ATGCCC 3' → 5' GGGCAT 3'

```
seq ← DNASTring("ATGCCC")  
  
reverseComplement(seq)  
# Output: GGGCAT
```

Clinical Note: Crucial when analyzing sequencing reads from the reverse strand.

Translating DNA to Protein

The Function

The `translate()` function applies the Standard Genetic Code to convert triplets of nucleotides into amino acids.

R Code

```
dna ← DNASTring("ATGAAACCCGGGTTTAA")  
  
protein ← translate(dna)  
  
# Output: AAString object  
# MKPGF*
```

M = Methionine (Start)

***** = Stop Codon

The Genetic Code

Biostrings has built-in lookup tables for the genetic code.

```
# View Standard Genetic Code
getGeneticCode("SGC0")

# Lookup specific codons
GENETIC_CODE["ATG"] # "M"
GENETIC_CODE["TAA"] # "*"
```

There are 64 possible codons coding for 20 amino acids plus stop signals.

Counting Nucleotides

```
seq ← DNASTring("ATGCATGCGGGCCC")  
  
# Count all bases  
alphabetFrequency(seq, baseOnly = TRUE)
```

A	C	G	T
2	5	4	3

Calculating GC Content

Why it matters

GC content affects melting temperature (T_m) and is a key metric in PCR primer design and genome characterization.

Calculation

```
seq ← DNASTring("ATGCATGCGGGCCC")

# Shortcut function
sum(letterFrequency(seq,
                    letters = c("G", "C"),
                    as.prob = TRUE)) * 100

# Output: 64.3%
```


Biological Sequence File Formats

Format	Extension	Contains	R Function
FASTA	.fasta, .fa	Sequence only	readDNASTringSet()
FASTQ	.fastq, .fq	Sequence + Quality	readDNASTringSet(format="fastq")
AB1	.ab1	Sanger Chromatogram	sangeranalyseR package

The FASTA Format

Simple, text-based format for representing nucleotide or peptide sequences.

```
>Sequence_Name_1  
ATGCATGCATGCATGCATGCATGCATGC  
ATGCATGCATGCATGC  
>Sequence_Name_2  
GCTAGCTAGCTAGCTAGCTAGCTAGCTA
```

- Header line starts with >.
- Sequence follows on next line(s).

Reading FASTA Files

```
# Read all sequences from FASTA file sequences <- readDNASTringSet("variants.fasta") # Access properties
length(sequences) # Number of sequences names(sequences) # Headers (names) width(sequences) # Length of each
sequence # Access specific sequence sequences[[1]] # First sequence
```

Writing FASTA Files

After processing data (e.g., trimming, filtering), you often need to save it.

```
# Write sequences to a new FASTA file  
writeXStringSet(sequences, "clean_output.fasta")
```

This is useful for exporting data to other tools (like BLAST or alignment software).

The FASTQ Format

FASTA + Quality Scores (used in NGS).

```
@Sequence_ID  
ATGCATGCATGC  
+  
IIIIIIIIIIII
```

- Header starts with @.
- + separates sequence from quality.
- Quality symbols represent probability of error (Phred score).

Reading FASTQ Files

```
# Standard read (ignores quality) fastq <- readDNAStringSet("reads.fastq", format = "fastq") # Reading with  
Quality (Requires ShortRead package) library(ShortRead) fq <- readFastq("reads.fastq") quality(fq) #  
Extract quality scores
```

Extracting Subsequences

Extracting specific regions (e.g., a gene within a genome).

```
seq ← DNASTring("ATGCDEFGHIJKLMNOP")  
  
# Extract bases 1 to 10  
subseq(seq, start = 1, end = 10)  
  
# Works on sets too  
subseq(sequences, start = 1, end = 100)
```

Finding Patterns (Motifs)

```
seq ← DNASTring("ATGAAATGCATGAAA")  
  
# Find all occurrences of "ATG" (Start codon)  
matchPattern("ATG", seq)
```

```
Views on a 15-letter DNASTring subject  
start end width  
[1] 1 3 3 [ATG]  
[2] 7 9 3 [ATG]  
[3] 12 14 3 [ATG]
```


Other Molecular Types

RNAString

```
dna ← DNASTring("ATG ... ")  
rna ← RNAString(dna)  
# T is replaced by U
```

AAString (Protein)

```
prot ← AAString("MKPGFTV")  
# Alphabet: 20 amino acids + stops
```

Clinical Lab Applications

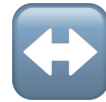
Application	Biostrings Use
Variant Detection	Align patient sequence vs reference to find mutations.
Genotyping	Locate SNPs in sequence data.
Species ID	Compare 16S rRNA sequences to databases.
Primer Design	Check GC content and melting temps.

Common Mistakes to Avoid



Alphabet

Using DNA functions on RNA objects (or vice versa).



Direction

Forgetting to Reverse Complement when analyzing the opposite strand.



Frame

Translating without checking the Open Reading Frame (ORF).

Key Takeaways

- ✓ **Bioconductor** is the hub for R bioinformatics.
- ✓ Use **BiocManager** to install packages.
- ✓ **DNAString/Set** classes handle sequences efficiently.
- ✓ Essential functions: `readDNAStringSet`, `translate`, `reverseComplement`.
- ✓ **FASTA** is the standard format for sequences.

Tutorial 9

Sequence Manipulation in Practice

Open your R environment

Tutorial 9: Sequence Manipulation

Working with SARS-CoV-2 Variant Sequences

Your Scenario

The Task

You are analyzing **SARS-CoV-2** variants.

You have sequences for:

- Wuhan-Hu-1 (Reference)
- Alpha (B.1.1.7)
- Delta (B.1.617.2)
- Omicron (BA.1)

Step 1: Create FASTA Data

We will generate a FASTA file containing simplified Spike gene sequences.

```
fasta_content <- ">Wuhan-Hu-1
ATGTTTGTTTTCTTGTTTTATTGCCACTAGTCTCTAG ...
>Alpha_B.1.1.7
ATGTTTGTTTTCTTGTTTTATTGCCACTAGTCTCTAG ...
... "

writeLines(fasta_content, "sars_cov2_variants.fasta")
```


Step 2: Read the FASTA File

Use the Biostrings function to load the data.

```
# TODO: Fill in the blank  
sequences ← readDNASTringSet(" ... ")
```

Check: You should have an object of length 4.

Step 3: Explore Properties

Inspect your data object.

```
length(sequences)  # How many?  
names(sequences)   # Which variants?  
width(sequences)   # How long?
```

Step 4: Calculate GC Content

Calculate the percentage of G and C nucleotides.

```
# TODO: Fill in letters
gc_freq <- letterFrequency(sequences,
                           letters = c(" ... ", " ... "),
                           as.prob = TRUE)

# Convert to percent
gc_percent <- rowSums(gc_freq) * 100
```

Step 5: Translate to Protein

Convert the DNA sequences to Amino Acid sequences.

```
# TODO: Translate
proteins ← translate( ... )

# View the result
print(proteins)
```

This allows us to see if mutations (DNA changes) result in amino acid changes.

Step 6: Compare Positions

Extract a specific nucleotide to see differences.

```
# Extract nucleotide at position 50  
subseq(sequences, start=50, end=50)
```

Does the Omicron variant differ from the Wuhan reference at this position?

Session 9 Complete

You have successfully entered the world of
Bioconductor!